

BEE 4750 Homework 4: Linear Programming and Capacity Expansion

Name: Evan Wu

ID: 5839048

Due Date

Thursday, 11/06/25, 9:00pm

Overview

Instructions

- Problem 1 asks you to analytically (or graphically) solve a linear program.
- Problem 2 asks you to formulate and solve a resource allocation problem using linear programming.
- Problem 3 asks you to formulate, solve, and analyze a standard generating capacity expansion problem.

Load Environment

The following code loads the environment and makes sure all needed packages are installed. This should be at the start of most Julia scripts.

```
import Pkg
Pkg.activate(@__DIR__)
Pkg.instantiate()
```

```

Activating project at `c:\Users\evan\OneDrive\Documents\GitHub\hw4-evetw`
Warning: The active manifest file has dependencies that were resolved with a different julia
@ nothing C:\Users\evan\OneDrive\Documents\GitHub\hw4-evetw\Manifest.toml:0
Warning: The project dependencies or compat requirements have changed since the manifest was
It is recommended to `Pkg.resolve()` or consider `Pkg.update()` if necessary.
@ Pkg.API C:\Users\evan\.julia\juliaup\julia-1.12.1+0.x64.w64.mingw32\share\julia\stdlib\v1.12
  Installed DisplayAs      v0.1.6
  Installed MarkdownTables v1.1.0
Precompiling packages...
      MbedTLS
7531.4 ms ArgCheck
7479.8 ms DisplayAs
2959.8 ms BenchmarkTools
      HTTP
6512.9 ms MarkdownTables
31114.7 ms PrettyTables
71202.5 ms MathOptInterface
6231.9 ms MathOptIIS
26309.0 ms HiGHS
83483.3 ms DataFrames
4775.3 ms Latexify → DataFramesExt
112032.1 ms Plots
45726.5 ms JuMP
12 dependencies successfully precompiled in 143 seconds. 207 already precompiled.

```

```

using JuMP
using HiGHS
using DataFrames
using Plots
using Measures
using CSV
using MarkdownTables

```

Problems (Total: 30 Points)

Problem 1 (Total: 3 Points)

Solve the following linear program **analytically**. Show your work and justify any reasoning.

$$\begin{aligned}
& \max_{x,y} \quad 3x + 2y \\
& \text{subject to:} \quad x + 4y \leq 16 \\
& \quad \quad \quad x - 2y \leq -1 \\
& \quad \quad \quad 0 \leq x \leq 4 \\
& \quad \quad \quad 1 \leq y \leq 4.
\end{aligned}$$

Goal is to maximize $3x + 2y$ based on the constraints above.

The constraints can be rearranged to let x be the subject:

$$\begin{aligned}
x &\leq 16 - 4y \\
x &\leq -1 + 2y \\
0 &\leq x \leq 4 \\
1 &\leq y \leq 4
\end{aligned}$$

This equation can be solved by finding points where the boundaries of different constraints meet and determining which yields the largest value from $3x + 2y$. Since $3x + 2y$ should be as large as possible, one corner point can be when $x = 4$.

$$\begin{aligned}
4 &\leq 16 - 4y \\
y &= 3
\end{aligned}$$

Therefore, y cannot be larger than 3. When $y = 1$:

$$\begin{aligned}
x &\leq -1 + 2 \\
x &= 1
\end{aligned}$$

We can check which value of y yields the maximum possible value by finding the intersection point between constraints 1 and 2.

Point C: From intersection of Constraints 1 and 2:

$$\begin{aligned}
x + 4y &= 16 \\
x &= -1 + 2y \\
-1 + 2y + 4y &= 16 \\
6y &= 17 \\
y &\approx 2.83
\end{aligned}$$

$$x \approx 4.67$$

Since the optimal value of y yields to a x value out of domain, the optimal value provided by the constraints would be $x = 4$, therefore $y = 3$.

The maximum of the objective function would be:

$$3 \times 4 + 2 \times 3 = 18$$

Problem 2 (12 points)

A farmer has access to a pesticide which can be used on corn, soybeans, and wheat fields, and costs \$70/kg-yr to apply. The crop yields the farmer can obtain following crop yields by applying varying rates of pesticides to the field are shown in Table 1.

Application Rate (kg/ha)	Soybean (kg/ha)	Wheat (kg/ha)	Corn (kg/ha)
0	2900	3500	5900
1	3800	4100	6700
2	4400	4200	7900

Table 1: Crop yields from applying varying pesticide rates for Problem 1.

The costs of production, *excluding pesticides*, for each crop, and selling prices, are shown in Table 2.

Crop	Production Cost (\$/ha-yr)	Selling Price (\$/kg)
Soybeans	350	0.36
Wheat	280	0.27
Corn	390	0.22

Table 2: Costs of crop production, excluding pesticides, and selling prices for each crop.

Recently, environmental authorities have declared that farms cannot have an *average* application rate on soybeans, wheat, and corn which exceeds 0.8, 0.7, and 0.6 kg/ha, respectively. The farmer has asked you for advice on how they should plant crops and apply pesticides to maximize profits over 130 total ha while remaining in regulatory compliance if demand for each crop (which is the maximum the market would buy) this year is 250,000 kg?

Problem 2.1

Formulate a linear program for this resource allocation problem, including clear definitions of decision variable(s) (including units), objective function(s), and constraint(s) (make sure to explain functions and constraints with any needed derivations and explanations). **Tip: Make sure that all of your constraints are linear.**

Goal is to maximize profits for selling the crops.

The decision variables would be the various rates for different pesticides as well as the total land used for each crop. - Let C = the crop with subscripts s,w and c for soybean, wheat and corn respectively - Let r = the pesticide rate with 0,1,2 for its corresponding rate in kg/ha.

The input variables for this problem would be: - A = hectares of land used for an individual crop C in ha - P_e = mass of pesticide used in kg

Constant variables: - Pesticide cost: 70 dollars per kg - Soybean yield: 2900 kg/ha for $r = 0$, 3800 kg/ha for $r = 1$, 4400 kg/ha for $r = 2$ - Wheat yield: 3500 kg/ha for $r = 0$, 4100 kg/ha for $r = 1$, 4200 kg/ha for $r = 2$ - Corn yield: 5900 kg/ha for $r = 0$, 6700 kg/ha for $r = 1$, 7900 kg/ha for $r = 2$ - Production costs and selling price listed in Table 2

The objective would be the maximum profit.

Maximum profit = Total revenue - Production costs - Pesticide cost

The constraints would be the maximum demand of crops, maximum application rates, and the land availability:

$$m_s \leq 250,000kg$$

$$m_w \leq 250,000kg$$

$$m_c \leq 250,000kg$$

$$(r_0 + r_1 + r_2) \times A_{s,r} \leq 0.8$$

$$(r_0 + r_1 + r_2) \times A_{w,r} \leq 0.7$$

$$(r_0 + r_1 + r_2) \times A_{c,r} \leq 0.6$$

$$\Sigma A \leq 130$$

Additionally, there cannot be any negative values for area:

$$A_{c,r} \geq 0$$

Total revenue and production costs must be expressed in terms of the total mass of each crop produced. Total mass of each crop produced is:

$$M_s = 2900 \times A_{s,0} + 3800 \times A_{s,1} + 4400 \times A_{s,2}$$

$$M_w = 3500 \times A_{w,0} + 4100 \times A_{w,1} + 4200 \times A_{w,2}$$

$$M_c = 5900 \times A_{c,0} + 6700 \times A_{s,1} + 7900 \times A_{c,2}$$

Total profit would be:

$$z = (0.36M_s + 0.27M_w + 0.22M_c) - (350 \times \Sigma A_s + 280 \times \Sigma A_w + 390 \times \Sigma A_c) - 70Pe$$

Problem 2.2

How many ha should the farmer dedicate to each crop and with what pesticide application rate(s)? How much profit will the farmer expect to make?

To implement the JuMP code for this linear program, two vectors containing arrays for each crop and references to each pesticide rate is created. Afterwards, dictionaries are created to link each crop type with a pesticide rate to correspond to certain material yields, prices, and costs. An additional dictionary is made

After, the model is created based on the HiGHS optimizer. The variables would be the areas used for all possible combinations of crop types and pesticide rates. The constraints mentioned previously are coded as follows: - Constraint to check that the sum of area used does not exceed the maximum at 130 ha. - Constraint to check that the sum of material generated from each crop type does not exceed 250,000 - Constraint to check that the average application of pesticides does not exceed environmental regulations.

Lastly, the objective function is created based on revenue - production costs - pesticide costs

```
# Indices
crops = [:soy, :wheat, :corn]
rates = [0, 1, 2]

# Yields (kg/ha)
y = Dict{
    (:soy, 0) => 2900, (:soy, 1) => 3800, (:soy, 2) => 4400,
    (:wheat, 0) => 3500, (:wheat, 1) => 4100, (:wheat, 2) => 4200,
    (:corn, 0) => 5900, (:corn, 1) => 6700, (:corn, 2) => 7900
}

# Prices and costs
price = Dict{:soy=>0.36, :wheat=>0.27, :corn=>0.22}
cost_ha = Dict{:soy=>350, :wheat=>280, :corn=>390} # $/ha without pesticide
rmax = Dict{:soy=>0.8, :wheat=>0.7, :corn=>0.6} # max avg application (kg/ha)

demand = Dict{:soy=>250000, :wheat=>250000, :corn=>250000} # kg
```

```

farm_model = Model(HiGHS.Optimizer)

@variable(farm_model, A[crops, rates] >= 0); # A[c,r] = hectares of crop c planted at applica
@constraint(farm_model, sum(A[c,r] for c in crops, r in rates) <= 130); # Land availability

for c in crops # Demand constraints
    @constraint(farm_model, sum(y[(c,r)] * A[c,r] for r in rates) <= demand[c]);
end

for c in crops # Average application-rate constraints
    @constraint(farm_model, sum((r - rmax[c]) * A[c,r] for r in rates) <= 0);
end

# Objective: maximize profit = revenue - production costs - pesticide cost
@objective(farm_model, Max, sum((price[c] * y[(c,r)] - cost_ha[c] - 70 * r) * A[c,r] for c in crops, r in rates));

# Solve
optimize!(farm_model);
@show value.(A)

```

Running HiGHS 1.11.0 (git hash: 364c83a51e): Copyright (c) 2025 HiGHS under MIT licence terms

LP has 7 rows; 9 cols; 27 nonzeros

Coefficient ranges:

Matrix	[2e-01, 8e+03]
Cost	[7e+02, 1e+03]
Bound	[0e+00, 0e+00]
RHS	[1e+02, 2e+05]

Presolving model

7 rows, 9 cols, 27 nonzeros 0s

7 rows, 9 cols, 27 nonzeros 0s

Presolve : Reductions: rows 7(-0); columns 9(-0); elements 27(-0) - Not reduced

Problem not reduced by presolve: solving the LP

Using EKK dual simplex solver - serial

Iteration	Objective	Infeasibilities	num(sum)
0	-1.8170298458e+02	Ph1: 7(28.3601); Du: 9(181.703)	0s
7	1.1674116702e+05	Pr: 0(0)	0s

Model status : Optimal

Simplex iterations: 7

Objective value : 1.1674116702e+05

P-D objective error : 0.0000000000e+00

HiGHS run time : 0.00

value.(A) = 2-dimensional DenseAxisArray{Float64,2,...} with index sets:

```

    Dimension 1, [:soy, :wheat, :corn]
    Dimension 2, [0, 1, 2]
And data, a 3×3 Matrix{Float64}:
 13.812154696132593  55.24861878453038   0.0
  6.743306417339564  15.73438164045898   0.0
 26.923076923076927   0.0                11.538461538461554

2-dimensional DenseAxisArray{Float64,2,...} with index sets:
  Dimension 1, [:soy, :wheat, :corn]
  Dimension 2, [0, 1, 2]
And data, a 3×3 Matrix{Float64}:
 13.8122  55.2486  0.0
  6.74331 15.7344  0.0
 26.9231  0.0     11.5385

```

```

# Indices
crops = [:soy, :wheat, :corn]
rates = [0, 1, 2]

# Yields (kg/ha)
y = Dict{
    (:soy, 0) => 2900, (:soy, 1) => 3800, (:soy, 2) => 4400,
    (:wheat, 0) => 3500, (:wheat, 1) => 4100, (:wheat, 2) => 4200,
    (:corn, 0) => 5900, (:corn, 1) => 6700, (:corn, 2) => 7900
}

# Prices and costs
price = Dict{:soy=>0.36, :wheat=>0.27, :corn=>0.22}
cost_ha = Dict{:soy=>350, :wheat=>280, :corn=>390} # $/ha without pesticide
rmax = Dict{:soy=>0.8, :wheat=>0.7, :corn=>0.6} # max avg application (kg/ha)

demand = Dict{:soy=>250000, :wheat=>250000, :corn=>250000} # kg

farm_model = Model(HiGHS.Optimizer)

@variable(farm_model, A[crops, rates] >= 0); # A[c,r] = hectares of crop c planted at applica
@constraint(farm_model, sum(A[c,r] for c in crops, r in rates) <= 130); # Land availability

for c in crops # Demand constraints
    @constraint(farm_model, sum(y[(c,r)] * A[c,r] for r in rates) <= demand[c]);
end

```



```

for c in crops # Average application-rate constraints
    @constraint(farm_model, sum((r - rmax[c]) * A[c,r] for r in rates) <= 0);
end

# Objective: maximize profit = revenue - production costs - pesticide cost
@objective(farm_model, Max, sum((price[c] * y[(c,r)] - cost_ha[c] - 70 * r) * A[c,r] for c in crops));

# Solve
optimize!(farm_model);
@show value.(A)

```

Running HiGHS 1.11.0 (git hash: 364c83a51e): Copyright (c) 2025 HiGHS under MIT licence terms

LP has 7 rows; 9 cols; 27 nonzeros

Coefficient ranges:

Matrix [2e-01, 8e+03]

Cost [7e+02, 1e+03]

Bound [0e+00, 0e+00]

RHS [1e+02, 2e+05]

Presolving model

7 rows, 9 cols, 27 nonzeros 0s

7 rows, 9 cols, 27 nonzeros 0s

Presolve : Reductions: rows 7(-0); columns 9(-0); elements 27(-0) - Not reduced

Problem not reduced by presolve: solving the LP

Using EKK dual simplex solver - serial

Iteration	Objective	Infeasibilities	num(sum)
0	-1.8170298458e+02	Ph1: 7(28.3601); Du: 9(181.703)	0s
7	1.1674116702e+05	Pr: 0(0)	0s

Model status : Optimal

Simplex iterations: 7

Objective value : 1.1674116702e+05

P-D objective error : 0.0000000000e+00

HiGHS run time : 0.00

value.(A) = 2-dimensional DenseAxisArray{Float64,2,...} with index sets:

Dimension 1, [:soy, :wheat, :corn]

Dimension 2, [0, 1, 2]

And data, a 3x3 Matrix{Float64}:

13.812154696132593	55.24861878453038	0.0
6.743306417339564	15.73438164045898	0.0
26.923076923076927	0.0	11.538461538461554

2-dimensional DenseAxisArray{Float64,2,...} with index sets:

Dimension 1, [:soy, :wheat, :corn]

```

    Dimension 2, [0, 1, 2]
And data, a 3×3 Matrix{Float64}:
13.8122  55.2486  0.0
 6.74331 15.7344  0.0
26.9231  0.0     11.5385

```

Problem 2.3

The farmer has an opportunity to buy an extra 10 ha of land. How much extra profit would this land be worth to the farmer? Discuss why this value makes sense and under what conditions you would recommend the farmer should make the purchase.

We modify the linear program from above by increasing the total land to 140 ha.

```

# Indices
crops = [:soy, :wheat, :corn]
rates = [0, 1, 2]

# Yields (kg/ha)
y = Dict{
    (:soy, 0) => 2900, (:soy, 1) => 3800, (:soy, 2) => 4400,
    (:wheat, 0) => 3500, (:wheat, 1) => 4100, (:wheat, 2) => 4200,
    (:corn, 0) => 5900, (:corn, 1) => 6700, (:corn, 2) => 7900
}

# Prices and costs
price = Dict{:soy=>0.36, :wheat=>0.27, :corn=>0.22}
cost_ha = Dict{:soy=>350, :wheat=>280, :corn=>390} # $/ha without pesticide
rmax = Dict{:soy=>0.8, :wheat=>0.7, :corn=>0.6} # max avg application (kg/ha)

demand = Dict{:soy=>250000, :wheat=>250000, :corn=>250000} # kg

farm_model = Model(HiGHS.Optimizer)

@variable(farm_model, A[crops, rates] >= 0); # A[c,r] = hectares of crop c planted at applica
@constraint(farm_model, sum(A[c,r] for c in crops, r in rates) <= 140); # Land availability

for c in crops # Demand constraints
    @constraint(farm_model, sum(y[(c,r)] * A[c,r] for r in rates) <= demand[c]);
end

for c in crops # Average application-rate constraints
    @constraint(farm_model, sum((r - rmax[c]) * A[c,r] for r in rates) <= 0);

```

```

end

# Objective: maximize profit = revenue - production costs - pesticide cost
@objective(farm_model, Max, sum((price[c] * y[(c,r)] - cost_ha[c] - 70 * r) * A[c,r] for c in 1:3, r in 1:2))

# Solve
optimize!(farm_model);
@show value.(A)

```

Running HiGHS 1.11.0 (git hash: 364c83a51e): Copyright (c) 2025 HiGHS under MIT licence terms

LP has 7 rows; 9 cols; 27 nonzeros

Coefficient ranges:

Matrix [2e-01, 8e+03]

Cost [7e+02, 1e+03]

Bound [0e+00, 0e+00]

RHS [1e+02, 2e+05]

Presolving model

7 rows, 9 cols, 27 nonzeros 0s

7 rows, 9 cols, 27 nonzeros 0s

Presolve : Reductions: rows 7(-0); columns 9(-0); elements 27(-0) - Not reduced

Problem not reduced by presolve: solving the LP

Using EKK dual simplex solver - serial

Iteration	Objective	Infeasibilities num(sum)
0	-1.8170298458e+02	Ph1: 7(28.3601); Du: 9(181.703) 0s
7	1.2403516702e+05	Pr: 0(0) 0s

Model status : Optimal

Simplex iterations: 7

Objective value : 1.2403516702e+05

P-D objective error : 1.1732040935e-16

HiGHS run time : 0.00

value.(A) = 2-dimensional DenseAxisArray{Float64,2,...} with index sets:

Dimension 1, [:soy, :wheat, :corn]

Dimension 2, [0, 1, 2]

And data, a 3×3 Matrix{Float64}:

13.81215469613259	55.2486187845304	0.0
9.743306417339566	22.734381640458984	0.0
26.923076923076923	0.0	11.538461538461522

2-dimensional DenseAxisArray{Float64,2,...} with index sets:

Dimension 1, [:soy, :wheat, :corn]

Dimension 2, [0, 1, 2]

And data, a 3×3 Matrix{Float64}:

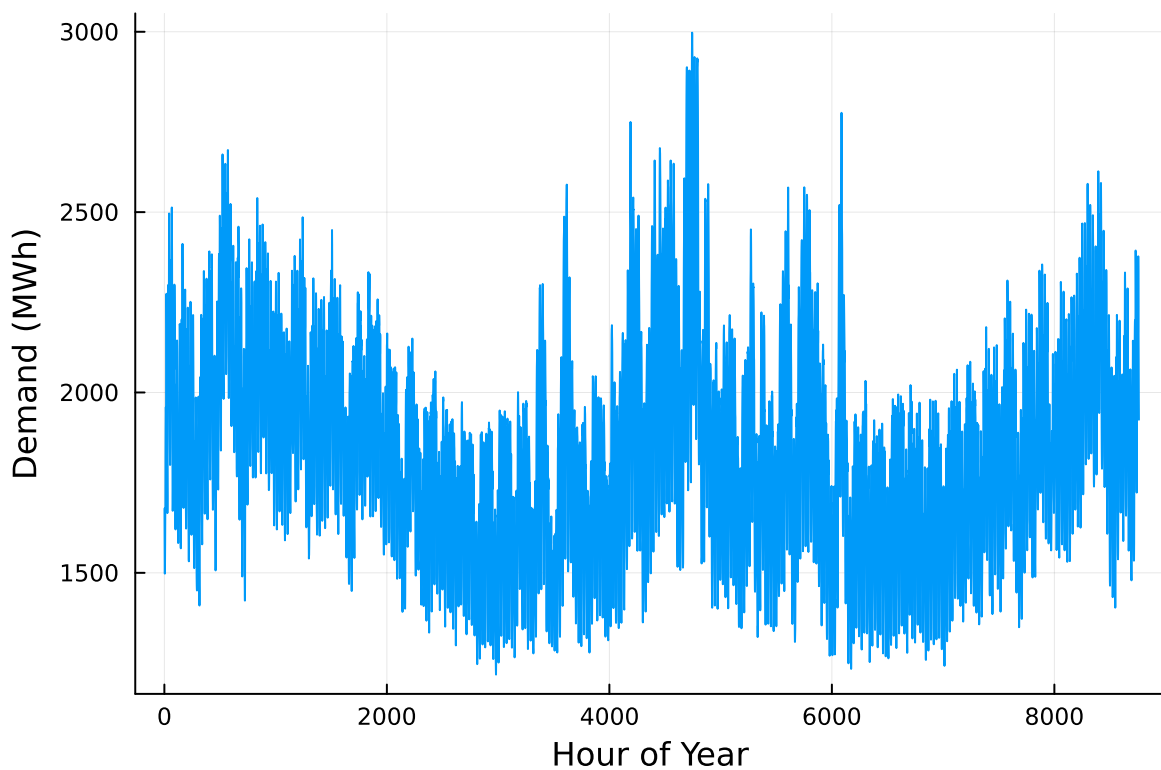
```
13.8122    55.2486    0.0
 9.74331   22.7344    0.0
26.9231     0.0     11.5385
```

Problem 3 (15 points)

For this problem, we will use hourly load (demand) data from 2013 in New York's Zone C (which includes Ithaca). The load data is loaded and plotted below in Figure 1.

```
# load the data, pull Zone C, and reformat the DataFrame
NY_demand = DataFrame(CSV.File("data/2013_hourly_load_NY.csv"))
rename!(NY_demand, :Time Stamp => :Date)
demand = NY_demand[:, [:Date, :C]]
rename!(demand, :C => :Demand)
demand[:, :Hour] = 1:nrow(demand)

# plot demand
plot(demand.Hour, demand.Demand, xlabel="Hour of Year", ylabel="Demand (MWh)", label=:false)
```



Next, we load the generator data, shown in Table 3. This data includes fixed costs (\$/MW installed), variable costs (\$/MWh generated), and CO2 emissions intensity (tCO2/MWh generated).

```
gens = DataFrame(CSV.File("data/generators.csv"))
```

	Plant	FixedCost	VarCost	Emissions
	String15	Int64	Int64	Float64
1	Geothermal	450000	0	0.0
2	Coal	220000	24	1.0
3	NG CCGT	82000	30	0.43
4	NG CT	65000	40	0.55
5	Wind	91000	0	0.0
6	Solar	70000	0	0.0

Finally, we load the hourly solar and wind capacity factors, which are plotted in Figure 2. These tell us the fraction of installed capacity which is expected to be available in a given hour for generation (typically based on the average meteorology).

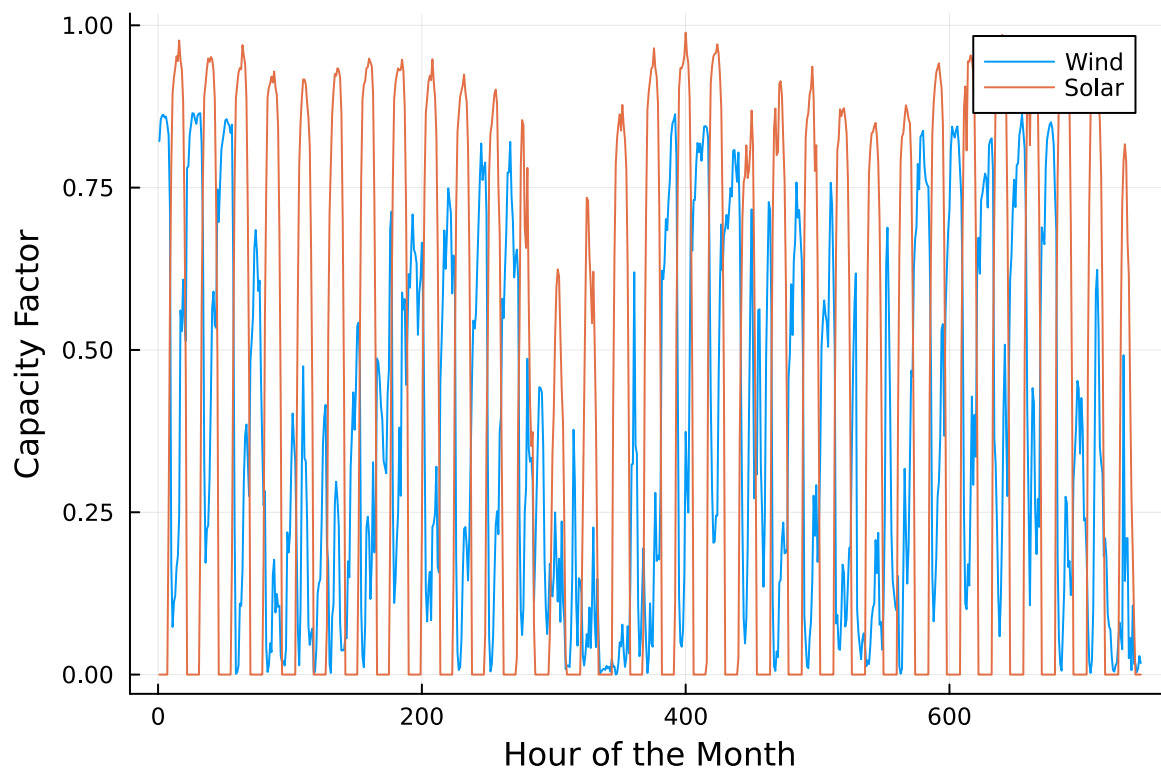
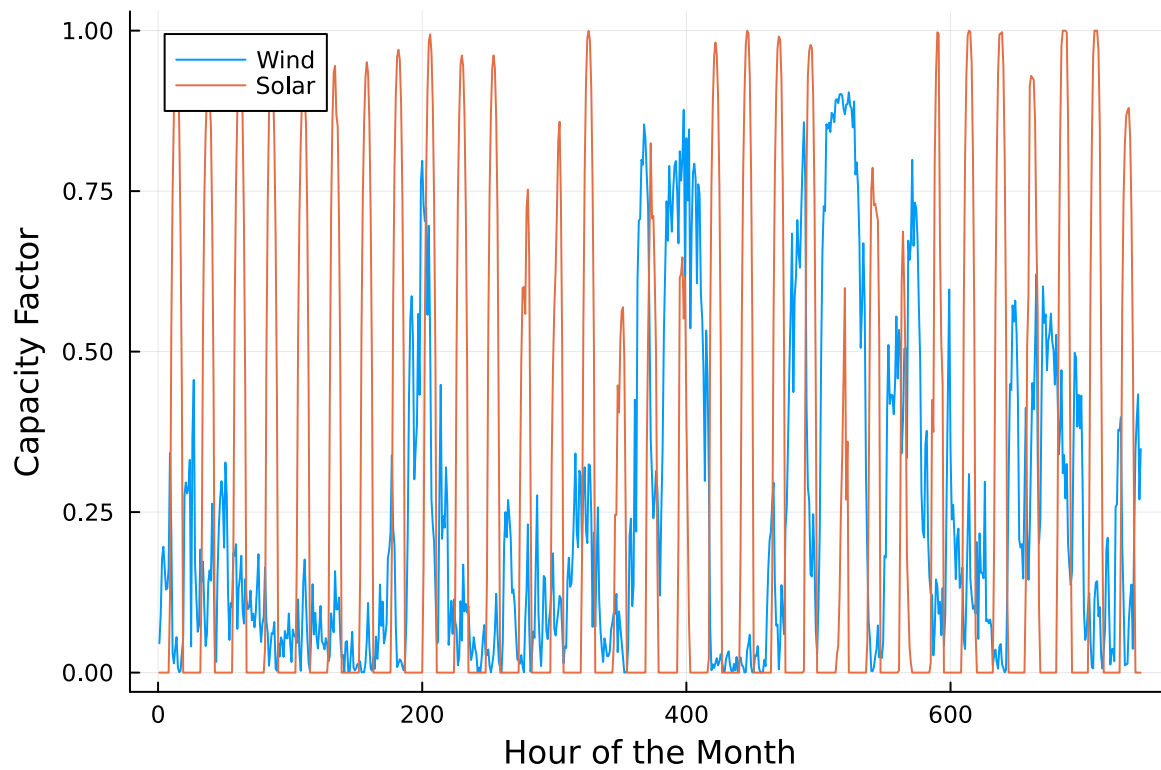
```
# load capacity factors into a DataFrame
cap_factor = DataFrame(CSV.File("data/wind_solar_capacity_factors.csv"))

# plot January capacity factors
p1 = plot(cap_factor.Wind[1:(24*31)], label="Wind")
plot!(cap_factor.Solar[1:(24*31)], label="Solar")
xaxis!("Hour of the Month")
yaxis!("Capacity Factor")

p2 = plot(cap_factor.Wind[4344:4344+(24*31)], label="Wind")
plot!(cap_factor.Solar[4344:4344+(24*31)], label="Solar")
xaxis!("Hour of the Month")
yaxis!("Capacity Factor")

display(p1)
display(p2)
```

You have been asked to develop a generating capacity expansion plan for the utility in Riley County, NY, which currently has no existing electrical generation infrastructure. The utility can build any of the following plant types: geothermal, coal, natural gas combined cycle gas turbine (CCGT), natural gas combustion turbine (CT), solar, and wind. The NY state legislature is planning to enact an annual CO2 limit, which for the utility would limit the emissions in its footprint to 1.5 MtCO2/yr.



While coal, CCGT, and CT plants can generate at their full installed capacity, geothermal plants operate at maximum 85% capacity, and solar and wind available capacities vary by the hour depend on the expected meteorology. The utility will also penalize any non-served demand at a rate of \$10,000/MWh.

Problem 3.1

Formulate a linear program for this capacity expansion problem, including clear definitions of decision variable(s) (including units), objective function(s), and constraint(s) (make sure to explain functions and constraints with any needed derivations and explanations).

Problem 3.2

How much should the utility build of each type of generating plant? What will the total cost be? How much energy will be non-served?

Problem 3.3

What fraction of annual generation does each plant type produce? How does this compare to the breakdown of built capacity that you found in Problem 3.2? Do these results make sense given the demand and generator data?

Problem 3.4

Make a plot of the electricity price in each hour. Discuss any trends that you see.

Problem 3.5

What is the shadow price of CO₂? What is the value to the utility be of allowing it to emit an additional 1000 tCO₂/yr?

Significant Digits

Use `round(x; digits=n)` to report values to the appropriate precision! If your number is on a different order of magnitude and you want to round to a certain number of significant digits, you can use `round(x; sigdigits=n)`.

Getting Variable Output Values

`value.(x)` will report the values of a JuMP variable `x`, but it will return a special container which holds other information about `x` that is useful for JuMP. This means that you can't use this output directly for further calculations. To just extract the values, use `value.(x).data`.

Suppressing Model Command Output

The output of specifying model components (variable or constraints) can be quite large for this problem because of the number of time periods. If you end a cell with an `@variable` or `@constraint` command, I *highly* recommend suppressing output by adding a semi-colon after the last command, or you might find that your notebook crashes.

References

List any external references consulted, including classmates.